

# **CHAPTER – 05**

**5.1 A CPU-scheduling algorithm determines an order for the execution of its scheduled processes. Given  $n$  processes to be scheduled on one processor, how many different schedules are possible? Give a formula in terms of  $n$ .**

The number of possible schedules for  $n$  processes on one processor is the same as the number of ways to arrange them. This is given by the formula  $n!$ , which means:

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 2 \times 1$$

So, the total number of different schedules for  $n$  processes is  $n!$ .

**5.2 Explain the difference between preemptive and non-preemptive scheduling.**

| Feature                     | Preemptive Scheduling                                           | Non-preemptive Scheduling                                     |
|-----------------------------|-----------------------------------------------------------------|---------------------------------------------------------------|
| <b>Definition</b>           | The CPU can be taken away from a running process.               | The CPU cannot be taken away from a running process.          |
| <b>Process Interruption</b> | A process can be interrupted mid-execution.                     | A process runs until it completes or blocks itself.           |
| <b>Context Switching</b>    | More frequent due to process interruption.                      | Less frequent, only occurs when a process finishes or blocks. |
| <b>Response Time</b>        | Typically lower, as high-priority tasks can be executed sooner. | Higher, as processes run to completion without interruptions. |
| <b>Use Case</b>             | Used in time-sharing systems where response fast is needed.     | Used in batch systems where processes can run uninterrupted.  |
| <b>Overhead</b>             | Higher, due to frequent context switching.                      | Lower, as there is less context switching.                    |
| <b>Example Algorithms</b>   | Round Robin, Shortest Remaining Time First (SRTF)               | First-Come-First-Served (FCFS), Shortest Job Next (SJN)       |

## 5.6 What advantage is there in having different time-quantum sizes at different levels of a multilevel queueing system?

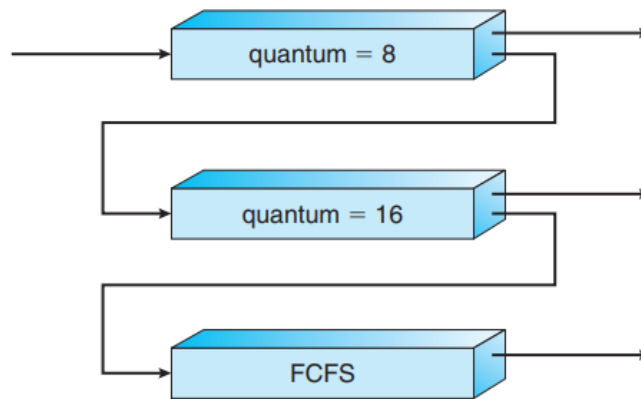


Figure 5.9 Multilevel feedback queues.

In a multilevel queueing system, using different time-quantum sizes at each level provides several benefits:

1. **Handling different types of processes:** Shorter time quanta are given to high-priority, interactive processes (like user input tasks), so they can be completed quickly and the system remains responsive. Longer time quanta are assigned to lower-priority, CPU-bound processes that need more continuous processing time, which helps them run efficiently without too many interruptions.
2. **Improved performance:** Interactive tasks can get fast response times, while longer tasks still get sufficient CPU time to complete. This balance improves overall system performance.
3. **Efficient CPU usage:** Short time quanta for quick tasks reduce delays, while longer time quanta for lower-priority tasks reduce the number of times the system has to switch between processes, minimizing the overhead from context switching.

This approach makes the system more efficient by prioritizing tasks based on their needs while ensuring all processes get CPU time.

**5.7 Many CPU-scheduling algorithms are parameterized. For example, the RR algorithm requires a parameter to indicate the time slice. Multilevel feedback queues require parameters to define the number of queues, the scheduling algorithms for each queue, the criteria used to move processes between queues, and so on. These algorithms are thus really sets of algorithms (for example, the set of RR algorithms for all time slices, and so on). One set of algorithms may include another (for example, the FCFS algorithm is the RR algorithm with an infinite time quantum). What (if any) relation holds between the following pairs of algorithm sets?\***

### **a. Priority and SJF**

- **Relation:** SJF is a special case of Priority.
- **Reason:** SJF assigns priority based on burst time (shorter jobs get higher priority).

### **b. Multi-level feedback queues and FCFS**

- **Relation:** FCFS is a special case of Multilevel Feedback Queues.
- **Reason:** If all queues in MLFQ use FCFS and there's no movement between queues, it acts like FCFS.

### **c. Priority and FCFS**

- **Relation:** No direct relation.
- **Reason:** FCFS schedules by arrival time, while Priority schedules by priority level.

### **d. RR and SJF**

- **Relation:** No direct relation.
- **Reason:** RR uses time slices, while SJF schedules by job length, aiming to minimize waiting time. Different goals.

**5.8 Suppose that a CPU scheduling algorithm favors those processes that have used the least processor time in the recent past. Why will this algorithm favor I/O-bound programs and yet not permanently starve CPU-bound programs?**

#### **Why the algorithm favors I/O-bound programs:**

1. **Low recent CPU usage:** I/O-bound processes use little CPU time.
2. **Frequent waiting:** They spend time waiting for I/O operations.
3. **Higher priority:** The scheduler gives them preference due to low CPU usage.

#### **Why CPU-bound programs are not permanently starved:**

1. **High initial CPU usage:** CPU-bound processes use more CPU time at first.
2. **Decreasing CPU usage over time:** As they wait, their recent CPU usage decreases.
3. **Priority increases:** The scheduler eventually gives them CPU time as their priority rises.

## 5.9 Distinguish between PCS and SCS scheduling.

| Feature                     | Process-Contention Scope (PCS)                                 | System-Contention Scope (SCS)                                |
|-----------------------------|----------------------------------------------------------------|--------------------------------------------------------------|
| <b>Scheduling Level</b>     | User-level (within the process)                                | Kernel-level (across all processes)                          |
| <b>Who Schedules</b>        | Thread library (user-level)                                    | Operating system (kernel)                                    |
| <b>Scope of Competition</b> | Threads compete within the same process                        | Threads compete across the entire system                     |
| <b>Visibility to OS</b>     | OS is unaware of user-level threads                            | OS is aware of all threads                                   |
| <b>Performance</b>          | More efficient, less overhead (user-level control)             | Less efficient due to kernel overhead                        |
| <b>Preemption</b>           | Managed by the user-level library, no OS preemption            | OS can preempt threads for better multitasking               |
| <b>Use Case</b>             | Used in many-to-one or many-to-many models                     | Used in one-to-one thread models                             |
| <b>Example</b>              | User-level threads in systems with lightweight processes (LWP) | Kernel-level threads competing for CPU resources system-wide |

This table summarizes the key differences between PCS and SCS, focusing on scheduling control, competition, and efficiency.

**5.10. The traditional UNIX scheduler enforces an inverse relationship between priority numbers and priorities: the higher the number, the lower the priority. The scheduler recalculates process priorities once per second using the following function:  $\text{Priority} = (\text{recent CPU usage} / 2) + \text{base}$  where  $\text{base} = 60$  and recent CPU usage refers to a value indicating how often a process has used the CPU since priorities were last recalculated. Assume that recent CPU usage for process P1 is 40, for process P2 is 18, and for process P3 is 10. What will be the new priorities for these three processes when priorities are recalculated? Based on this information, does the traditional UNIX scheduler raise or lower the relative priority of a CPU-bound process?**

Let's break down the calculation for each process using the priority function:

$$\text{Priority} = (\text{recent CPU usage} / 2) + \text{base}$$

**For Process P1:**

- Recent CPU usage = 40
- Priority (P1) =  $(40/2) + 60 = 20 + 60 = 80$

#### **For Process P2:**

- Recent CPU usage = 18
- Priority (P2) =  $(18/2) + 60 = 9 + 60 = 69$

#### **For Process P3:**

- Recent CPU usage = 10
- Priority (P3) =  $(10/2) + 60 = 5 + 60 = 65$

#### **New Priorities:**

- Process P1: Priority = 80
- Process P2: Priority = 69
- Process P3: Priority = 65

#### **Analysis:**

The traditional UNIX scheduler uses an inverse relationship between the priority number and the actual priority, meaning that a higher priority number corresponds to a lower priority. Since the priority increases with higher CPU usage (as shown in P1's case), the scheduler lowers the priority of CPU-bound processes. In this case:

- Process P1, being more CPU-bound (having higher recent CPU usage), has a higher priority number, meaning it has a **lower actual priority**.
- Process P3, with lower recent CPU usage, has a lower priority number, meaning it has a **higher actual priority**.

### **5.11 Of these two types of programs:**

#### **a. I/O –bound**

#### **b. CPU-bound**

**Which is more likely to have voluntary context switches, and which is more likely to have non-voluntary context switches? Explain your answer.**

Let's break down the answer by first understanding voluntary and non-voluntary context switches:

#### **Voluntary Context Switch:**

This occurs when a process voluntarily gives up the CPU, typically when it's waiting for some I/O operation to complete, such as reading from a file or waiting for user input.

### **Non-voluntary Context Switch:**

This happens when the operating system forcibly removes a process from the CPU, typically because its allocated time slice has expired (in a time-sharing system), or due to preemption by a higher-priority process.

#### **a. I/O-bound programs**

- **More likely to have voluntary context switches.**
- I/O-bound programs spend much of their time waiting for I/O operations to complete, such as reading from or writing to files, interacting with the network, or waiting for user input. Once they start an I/O operation, they voluntarily yield the CPU, allowing other processes to run while they wait for the I/O to finish.

#### **b. CPU-bound programs:**

- **More likely to have non-voluntary context switches.**
- CPU-bound programs use the CPU intensively and tend to execute without waiting for I/O. These programs are likely to use their entire time slice, meaning the operating system must forcibly preempt them (non-voluntary context switch) when their time slice ends or when a higher-priority process needs CPU time.

## **5.12 Discuss how the following pairs of scheduling criteria conflict in certain settings.**

### **a. CPU utilization and response time**

### **b. Average turnaround time and maximum waiting time**

### **c. I/O device utilization and CPU utilization**

| <b>Criteria Pair</b>                                    | <b>Explanation</b>                                                                                                                                                                                                                                                                      | <b>Conflict</b>                                                                                                                                                                                                                                |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CPU Utilization vs. Response Time</b>                | <ul style="list-style-type: none"><li>- <b>CPU Utilization:</b> Measures CPU efficiency, higher utilization means less idle time.</li><li>- <b>Response Time:</b> Time taken to start responding to a process request. Lower response time is better for interactive systems.</li></ul> | <ul style="list-style-type: none"><li>- Maximizing CPU utilization can cause delays for interactive tasks, increasing response time.</li><li>- Minimizing response time may reduce CPU utilization due to frequent context switches.</li></ul> |
| <b>Average Turnaround Time vs. Maximum Waiting Time</b> | <ul style="list-style-type: none"><li>- <b>Average Turnaround Time:</b> Time from process submission to completion (including waiting and</li></ul>                                                                                                                                     | <ul style="list-style-type: none"><li>- Reducing average turnaround time (e.g., prioritizing short jobs) can increase maximum waiting time for long-running</li></ul>                                                                          |

|                                                   |                                                                                                                                                                                      |                                                                                                                                                                                    |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                   | execution).<br>- <b>Maximum Waiting Time:</b><br>The longest time any process waits for CPU time. Reducing it ensures fairness.                                                      | jobs.<br>- Reducing maximum waiting time may increase average turnaround time.                                                                                                     |
| <b>I/O Device Utilization vs. CPU Utilization</b> | - <b>I/O Device Utilization:</b><br>Measures efficiency of I/O devices, aims to keep them busy.<br>- <b>CPU Utilization:</b> Measures CPU efficiency, aims to minimize idle periods. | - Focusing on CPU utilization may lead to I/O devices being idle.<br>- Focusing on I/O device utilization may lead to CPU being idle while waiting for I/O operations to complete. |

**5.13 One technique for implementing lottery scheduling works by assigning processes lottery tickets, which are used for allocating CPU time. Whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the CPU. The BTV operating system implements lottery scheduling by holding a lottery 50 times each second, with each lottery winner getting 20 milliseconds of CPU time ( $20 \text{ milliseconds} \times 50 = 1 \text{ second}$ ). Describe how the BTV scheduler can ensure that higher-priority threads receive more attention from the CPU than lower-priority threads.**

To ensure that higher-priority threads receive more CPU attention than lower-priority threads in lottery scheduling, the BTV scheduler can implement **priority-based ticket allocation**. Here's how it can work:

1. **Assign more lottery tickets to higher-priority threads:** In lottery scheduling, the number of tickets a process holds determines its chances of winning the lottery and getting CPU time. The BTV scheduler can assign a **higher number of tickets to higher-priority threads** and a **lower number of tickets to lower-priority threads**.
  - o For example, if there are two processes, one high-priority and one low-priority:
    - The high-priority process might receive 100 tickets.
    - The low-priority process might receive 10 tickets.
2. **Higher chances for high-priority threads to win:** Since the scheduler randomly selects a lottery ticket, processes with more tickets have a **higher probability of winning the lottery** and being allocated CPU time. This ensures that higher-priority threads win the lottery more often and thus get more CPU time.
3. **Fairness remains intact:** Lower-priority threads still have some tickets, so they aren't completely starved—they just win the lottery less frequently. This maintains a balance between prioritizing important tasks and ensuring that lower-priority tasks eventually receive some CPU time.

**Example:**

- Suppose the system runs 50 lotteries per second.
- If a high-priority thread holds 70% of the total tickets, it will win roughly 35 lotteries per second and get approximately  $35 \times 20$  milliseconds = **700 milliseconds** of CPU time.
- A lower-priority thread with 10% of the tickets will win about 5 lotteries per second, getting **100 milliseconds** of CPU time.

**5.14 Most scheduling algorithms maintain a run queue, which lists processes eligible to run on a processor. On multicore systems, there are two general options: (1) each processing core has its own run queue, or (2) a single run queue is shared by all processing cores. What are the advantages and disadvantages of each of these approaches?**

| Approach                               | Advantages                                                                                                                                      | Disadvantages                                                                                                                                                      |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Each core has its own run queue</b> | <ul style="list-style-type: none"> <li>- Reduced contention</li> <li>- Better cache locality</li> <li>- More scalable for many cores</li> </ul> | <ul style="list-style-type: none"> <li>- Potential for load imbalance</li> <li>- Requires complex load-balancing mechanisms</li> </ul>                             |
| <b>Single shared run queue</b>         | <ul style="list-style-type: none"> <li>- Ensures balanced load</li> <li>- Simpler scheduling logic</li> </ul>                                   | <ul style="list-style-type: none"> <li>- Increased contention for access</li> <li>- Reduces cache locality</li> <li>- Less scalable as core count grows</li> </ul> |

**5.15 Consider the exponential average formula used to predict the length of the next CPU burst. What are the implications of assigning the following values to the parameters used by the algorithm?**

**a.  $\alpha = 0$  and  $\tau_0 = 100$  milliseconds**

**b.  $\alpha = 0.99$  and  $\tau_0 = 10$  milliseconds**

The exponential average formula used to predict the length of the next CPU burst is:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n.$$

The value of  $\alpha$  determines how much weight is given to the most recent CPU burst time compared to the previous predicted burst time.

**a.  $\alpha = 0$  and  $\tau_0 = 100$  milliseconds**

- **Implications:**

- o When  $\alpha=0$ , the formula becomes:  $\tau_{n+1}=\tau_n$  This means that the prediction for the next CPU burst will always be the same as the initial prediction.
- o In this case, the system **completely ignores** the actual length of the current CPU burst and relies solely on the initial prediction.
- o **Prediction:** The next CPU burst will always be predicted as 100 milliseconds, regardless of actual behavior.
- o **Consequence:** The prediction will remain constant over time, which can lead to **inaccurate predictions** if the actual burst times deviate significantly from 100 milliseconds.

**b.  $\alpha = 0.99$  and  $\tau_0 = 10$  milliseconds**

- **Implications:**
  - o When  $\alpha=0.99$ , the formula heavily weights the most recent CPU burst time  $\tau_{n+1} \approx 0.99 \cdot \tau_n + 0.01 \cdot \tau_n$
  - o This means that the next predicted burst length is **almost entirely based on the most recent burst**. The initial prediction ( $\tau_0$ ) has very little influence after the first few bursts.
  - o **Prediction:** The next prediction will closely follow the actual CPU burst times, quickly adapting to recent behavior.
  - o **Consequence:** The prediction will be **highly responsive** to changes in CPU burst length but may result in **less stability** since it fluctuates significantly based on recent bursts.

**5.16 A variation of the round-robin scheduler is the regressive round-robin scheduler. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process (CPU-bound or I/O -bound) does the regressive round-robin scheduler favor? Explain.**

The **regressive round-robin scheduler** favors **CPU-bound processes**. Here's the explanation:

**How the Scheduler Works:**

1. **Time Quantum Increase:** Every time a process uses its entire time quantum (does not block for I/O), 10 milliseconds are added to the time quantum, and the process's priority is boosted. This happens until the time quantum reaches a maximum of 100 milliseconds.

2. **Time Quantum Decrease:** If a process blocks before using its entire time quantum (typical for I/O-bound processes), the time quantum is reduced by 5 milliseconds, but the priority remains unchanged.

#### **Impact on Process Types:**

- **CPU-bound processes** tend to use the entire time quantum since they are constantly executing instructions and do not wait for I/O operations. As a result, their time quantum gradually increases up to 100 milliseconds, and their priority is boosted. This gives CPU-bound processes more CPU time and higher priority, allowing them to execute longer without being preempted.
- **I/O-bound processes** frequently block for I/O before their time quantum expires. Each time this happens, their time quantum decreases by 5 milliseconds. Since I/O-bound processes rely on frequent I/O operations, they will often block before using their full quantum, leading to smaller and smaller quanta. This results in them receiving less CPU time.

#### **Conclusion:**

The regressive round-robin scheduler **favors CPU-bound processes** because it increases their time quantum and priority when they use their full time slice. In contrast, **I/O-bound processes** see their time quantum decrease when they block early, making them less favored for CPU time allocation.

**5.19 The nice command is used to set the nice value of a process on Linux, as well as on other UNIX systems. Explain why some systems may allow any user to assign a process a nice value  $\geq 0$  yet allow only the root (or administrator) user to assign nice values  $< 0$ .**

In Linux and UNIX systems, the **nice** value controls the priority of a process. A lower nice value gives a process higher priority, and a higher nice value gives it lower priority. Here's why only administrators can set nice values below 0:

#### **1. Regular users (nice value $\geq 0$ ):**

Regular users are allowed to set a nice value  $\geq 0$ , which lowers the priority of their processes. This prevents users from consuming too many CPU resources and ensures the system remains fair to all users.

#### **2. Administrator (nice value $< 0$ ):**

Only the **administrator** can assign a nice value  $< 0$ , which increases the priority of a process. This is restricted because allowing regular users to raise their process priority could lead to system imbalance, where their processes monopolize CPU time, potentially affecting other users and important system tasks.

## 5.20 Which of the following scheduling algorithms could result in starvation?

### a. First-come, first-served

- **No starvation.**
- Reason: Processes are scheduled in the order they arrive, so every process will eventually get the CPU.

### b. shortest job first

- **Yes, can cause starvation.**
- Reason: Shorter jobs are always prioritized. Long-running processes could be perpetually delayed if short processes keep arriving, leading to potential starvation for longer jobs.

### c. Round robin

- **No starvation.**
- Reason: Each process is given a fixed time slice in a rotating order, ensuring that every process gets CPU time at regular intervals, preventing starvation.

### d. Priority

- **Yes, can cause starvation.**
- Reason: Lower-priority processes may be indefinitely postponed if higher-priority processes continuously enter the system, causing starvation for low-priority tasks.

## 5.21 Consider a variant of the RR scheduling algorithm in which the entries in the ready queue are pointers to the PCBs. \*(3)

### a. What would be the effect of putting two pointers to the same process in the ready queue?

- ☐ The process will be scheduled **more frequently** than others.
- ☐ This results in the process receiving **twice the CPU time** in each Round Robin cycle.
- ☐ It essentially gives the process **higher priority** compared to other processes with only one pointer.

### b. What would be two major advantages and two disadvantages of this scheme?

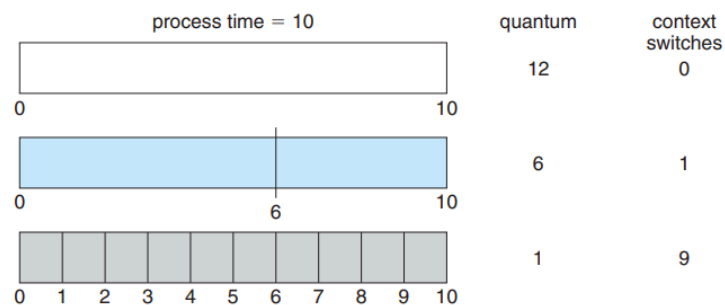
### Advantages:

1. It allows **important processes** to get more CPU time.
2. It gives **flexibility** in CPU time allocation without changing the entire algorithm.

### Disadvantages:

1. It can lead to **unfairness** by giving some processes more CPU time.
2. It increases **complexity** in managing the ready queue.

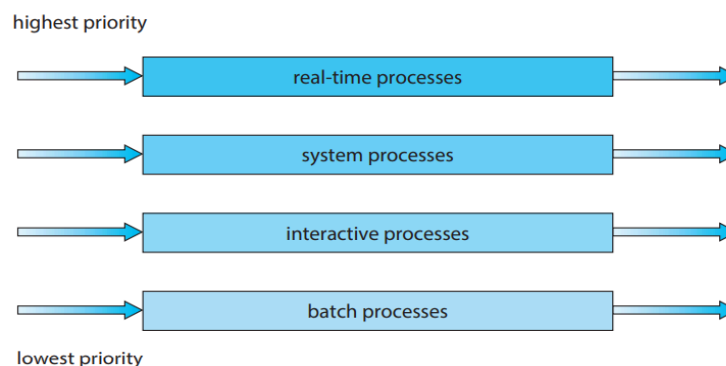
**c. How would you modify the basic RR algorithm to achieve the same effect without the duplicate pointers?**



**Figure 5.5** How a smaller time quantum increases context switches.

Use a **weighted time slice**, giving certain processes longer CPU time. This allows processes to run longer without adding extra pointers in the queue.

**5.23 Consider a system implementing multilevel queue scheduling. What strategy can a computer user employ to maximize the amount of CPU time allocated to the user's process?**



**Figure 5.8** Multilevel queue scheduling.

### 1. Make the Process Look Interactive or Real-Time:

- **Strategy:** Design the process to do small tasks and use I/O frequently.
- **Reason:** **Real-time and interactive processes** get **higher priority** in the system, so they get more CPU time.

## 2. Increase Process Priority:

- **Strategy:** Use tools like `nice` or `renice` to **increase the process priority**.
- **Reason:** A higher priority moves the process to a **higher-priority queue**, giving it more CPU time.

## 3. Avoid Batch-Process Behavior:

- **Strategy:** Don't make the process run for long, uninterrupted periods.
- **Reason:** **Batch processes** are placed in the **lowest-priority queue**, meaning they get less CPU time. To avoid this, keep the process short and interactive.

## 4. Keep the Process Active:

- **Strategy:** Ensure the process stays **active and responsive**.
- **Reason:** Processes that stay active are likely to remain in **higher-priority queues**, ensuring they get more CPU attention.

To get more CPU time, make your process behave like **real-time or interactive tasks**, increase its priority, avoid long-running batch behaviors, and keep it active. This will help the system assign more CPU time to your process.

**5.24 Consider a preemptive priority scheduling algorithm based on dynamically changing priorities. Larger priority numbers imply higher priority. When a process is waiting for the CPU (in the ready queue, but not running), its priority changes at a rate  $\alpha$ . When it is running, its priority changes at a rate  $\beta$ . All processes are given a priority of 0 when they enter the ready queue. The parameters  $\alpha$  and  $\beta$  can be set to give many different scheduling algorithms.**

### a. What is the algorithm that results from $\beta > \alpha > 0$ ?

A running process's priority increases faster than when it's waiting. Once a process starts running, its priority rises quickly, making it not to be preempted. Resulting Algorithm is First-Come, First-Served (FCFS).

### b. What is the algorithm that results from $\alpha < \beta < 0$ ?

Waiting processes lose priority faster than running ones. New processes enter with a priority of 0, higher than existing processes whose priorities have decreased, happened preemption. Resulting Algorithm is Last-In, First-Out (LIFO) or Preemptive Shortest Job First (SJF).

| Scenario                   | $\beta > \alpha > 0$                                                  | $\alpha < \beta < 0$                                                    |
|----------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Priority Changes</b>    | Priority increases faster when running than waiting                   | Priority decreases faster when waiting than running                     |
| <b>Effect on Processes</b> | Running processes gain priority quickly and tend to run to completion | New processes have higher priority; running processes are preempted     |
| <b>Resulting Algorithm</b> | <b>First-Come, First-Served (FCFS)</b>                                | <b>Last-In, First-Out (LIFO) or Preemptive Shortest Job First (SJF)</b> |
| <b>Scheduling Behavior</b> | Processes run in arrival order without preemption                     | Processes with less CPU time used are scheduled first                   |

## 5.25 Explain the how the following scheduling algorithms discriminate either in favor of or against short processes:

### a. FCFS

**No discrimination between short or long processes:** FCFS schedules processes strictly based on their arrival order, without considering their length. It treats all processes equally in terms of when they arrived, but this can result in **indirect discrimination against short processes** if a long process arrives before them. Short processes have to wait behind longer ones, even though they could finish quickly.

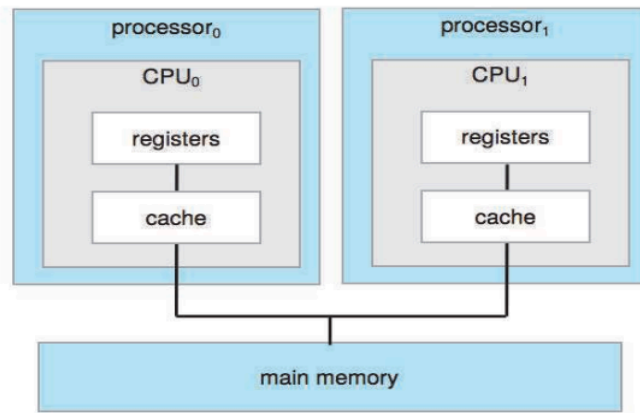
### b. RR

**Somewhat favors short processes:** RR gives each process a **fixed time slice**. Short processes are favored because they may complete within one or two time slices, while longer processes have to wait for multiple cycles to finish. If the time quantum is well-tuned (small enough), short processes get CPU access more frequently, allowing them to finish faster.

### c. multilevel feedback queues

**Favors short processes:** This algorithm **actively favors short processes**. Processes that use less CPU time remain in **higher-priority queues**, which get more frequent CPU attention. Long processes are gradually moved to **lower-priority queues** with longer time slices, ensuring that short processes get completed more quickly.

## 5.26 Describe why a shared ready queue might suffer from performance problems in an SMP environment.



**Figure 1.8** Symmetric multiprocessing architecture.

A **shared ready queue** in an **SMP environment** can cause performance problems due to:

- ❑ **Competition:** Multiple processors accessing the queue at the same time create bottlenecks, slowing down the system.
- ❑ **Cache Coherency:** Synchronizing the queue between processor caches increases delays.
- ❑ **Load Balancing:** Some processors might pick up lighter tasks, leaving others with heavier tasks, leading to inefficiency.
- ❑ **Locking Overhead:** Using locks to prevent race conditions adds extra processing time.
- ❑ **Scalability Issues:** As more processors are added, these problems get worse, reducing overall performance.

In short, the shared queue becomes a bottleneck when too many processors try to access it at once.

**5.27 Consider a load-balancing algorithm that ensures that each queue has approximately the same number of threads, independent of priority. How effectively would a priority-based scheduling algorithm handle this situation if one run queue had all high-priority threads and a second queue had all low-priority threads?**

In this scenario, a **priority-based scheduling algorithm** would likely handle the situation **ineffectively**. Here's why:

- ❑ **High-priority threads dominate:** The CPU will focus on the high-priority threads, giving them most of the time, while the low-priority threads will get much less attention.
- ❑ **Low-priority threads wait longer:** Low-priority threads in the second queue will experience long delays and may even be ignored (starvation) because the system keeps serving high-priority threads first.

- **Unbalanced load:** Even if both queues have the same number of threads, the CPU time will be unevenly distributed, with high-priority threads getting most of it, making load balancing ineffective.

The system won't handle this well because it focuses on priorities, not evenly distributing CPU time between the queues.

**5.28 Assume that an SMP system has private, per-processor run queues. When a new process is created, it can be placed in either the same queue as the parent process or a separate queue.**

**a. What are the benefits of placing the new process in the same queue as its parent?**

**b. What are the benefits of placing the new process in a different queue?**

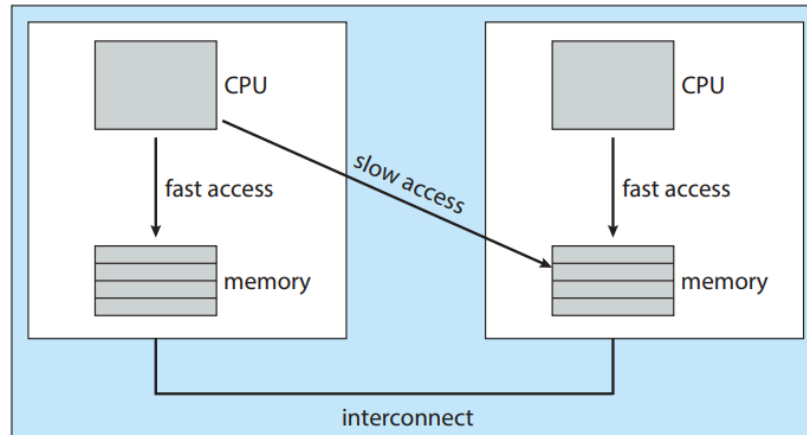
**a. Benefits of Placing the New Process in the Same Queue as Its Parent**

1. **Cache Affinity:** Increases chances of using data already in the processor's cache, improving performance.
2. **Simplified Load Balancing:** Reduces the need for load balancing between processors, simplifying scheduling.
3. **Faster Communication:** Reduces inter-processor communication overhead, speeding up process synchronization.
4. **Reduced Context Switching:** Lower overhead since processes may be scheduled back-to-back on the same processor

**b. Benefits of Placing the New Process in a Different Queue**

1. **Better Load Distribution:** Distributes workload evenly across processors, preventing overload on one core.
2. **Increased Parallelism:** Allows parent and child processes to run concurrently on different processors, speeding execution.
3. **Improved Scalability:** Enhances system scalability by better utilizing multiple processors in parallel.
4. **Avoiding Processor Contention:** Prevents competition for the same processor, ensuring smoother execution of both processes.

**5.29 Assume that a thread has blocked for network I/O and is eligible to run again. Describe why a NUMA-aware scheduling algorithm should reschedule the thread on the same CPU on which it previously ran.**



**Figure 5.16** NUMA and CPU scheduling.

- ☐ **Local Memory Access:** Rescheduling the thread on the same CPU allows it to access its local memory faster. Switching CPUs would result in slower memory access.
- ☐ **Cache Locality:** The same CPU retains cached data from previous runs, improving performance. Moving to a new CPU would require reloading data into the cache.
- ☐ **Performance Optimization:** Keeping the thread on the same CPU reduces delays from accessing non-local memory and reloading caches, ensuring better performance

**5.36 Explain why interrupt and dispatch latency times must be bounded in a hard real-time system.**

#### **1. Interrupt Latency:**

- **Definition:** Interrupt latency is the time taken by the system to respond to an interrupt, i.e., the time between when an interrupt is generated and when the operating system starts executing the interrupt service routine (ISR).

#### **Why it must be bounded:**

- In a hard real-time system, certain events (e.g., sensor input, emergency signals) need to be handled **immediately**. If interrupt latency is not bounded (i.e., unpredictable or too long), the system may not react to these time-sensitive events in time, leading to missed deadlines and potential failures.
- For example, in an airbag deployment system in a car, if the interrupt latency is too high, the airbag might deploy too late, causing injury or death.

## 2. Dispatch Latency:

- **Definition:** Dispatch latency is the time it takes for the operating system to stop one process and start or dispatch another (e.g., a high-priority real-time task).

### Why it must be bounded:

- In a hard real-time system, tasks often have strict deadlines. If dispatch latency (the time to switch to a high-priority task) is not bounded, the system may fail to start the required real-time task in time, causing it to miss its deadline.
- For example, in an industrial robot control system, if the dispatch latency is too long, the robot might fail to react to commands, leading to malfunction or damage to the product.

## 5.37 Describe the advantages of using heterogeneous multiprocessing in a mobile system.

Heterogeneous multiprocessing (HMP) in mobile systems involves using processors with different architectures and performance characteristics, typically a combination of high-performance cores and energy-efficient cores. This approach is used in systems like ARM's **big. LITTLE architecture**, where "big" cores are powerful and energy-intensive, while "LITTLE" cores are energy-efficient but slower.

Advantages of Using Heterogeneous Multiprocessing in a Mobile System:

- ❑ **Improved Power Efficiency:** Dynamically assigns tasks to high-performance or energy-efficient cores based on workload, reducing power consumption and extending battery life.
- ❑ **Performance Optimization:** High-performance cores handle demanding tasks, ensuring smooth execution of CPU-bound operations without straining energy-efficient cores.
- ❑ **Thermal Management:** Offloads less critical tasks to energy-efficient cores, reducing heat generation and preventing thermal throttling for stable performance.
- ❑ **Extended Battery Life:** Energy-efficient cores handle low-power tasks, reducing overall power consumption and extending battery life.
- ❑ **Adaptability to Varying Workloads:** Efficiently switches between cores based on task requirements, optimizing both performance and power for different types of workloads.
- ❑ **Cost Efficiency:** Reduces power consumption for low-priority tasks, offering energy efficiency and high performance without the need for expensive cooling solutions.

